

智能题库刷题助手

项目总结报告

C++ 数据结构与算法编程 • 结课作业

计算机与信息技术学院 • 大一

2026 年 6 月

陈彦邹

目录

1	项目概述	3
1.1	项目背景	3
1.2	技术选型	3
1.3	项目规模	3
2	AI 迭代对话记录	4
2.1	需求分析与架构设计	4
2.2	开发过程中的代码审查与修复	5
3	核心算法分析	7
3.1	HTTP 路由分发算法	7
3.2	题目随机打乱算法 (Fisher-Yates Shuffle)	7
3.3	答案比对算法 (大小写不敏感 + 去空白)	8
3.4	SQL 动态查询构建算法	8
4	数据结构应用分析	10
4.1	核心数据结构一览	10
4.2	课程知识点对照	10
5	工程能力提升总结	12
5.1	AI 辅助开发的经验与反思	12
5.1.1	AI 生成代码的优点	12
5.1.2	AI 生成代码的局限性	12
5.1.3	人工算法的必要性	12
5.2	个人能力提升	12
6	项目总结	14
6.1	完成情况	14
6.2	工程反思	14
6.3	致谢	14

1 项目概述

1.1 项目背景

本项目是为 C++ 数据结构与算法编程课程开发的结课项目——一个完整的智能题库刷题系统。系统基于 C++17 开发高性能 HTTP 服务端，结合单文件 HTML 前端，支持多模式刷题、错题管理、AI 智能辅助等功能。

学生在日常学习中面临题库管理混乱、刷题效率低下、错题难以追踪等痛点。本项目通过编程方式系统性地解决了这些问题。

1.2 技术选型

层级	技术	说明
后端服务	C++17 + cpp-httplib	860 行，18 个 REST API 端点
数据库	SQLite3 (amalgamation)	C++ RAII 封装，预编译语句
AI 服务	libcurl + nlohmann/json	RAII 句柄管理，多厂商兼容
前端	Bootstrap 5 + Chart.js	单文件 SPA，2144 行
文档	L ^A T _E X / Markdown	设计文档 + 本报告

表 1: 技术选型总览

1.3 项目规模

- C++ 源码: 2,100 行 (quiz_server.cpp, database.cpp, ai_service.cpp)
- HTML 前端: 2,150 行 (单文件 SPA, 7 个功能页面)
- API 端点: 18 个 RESTful 接口
- 数据库表: 4 张 (categories, questions, user_records, wrong_questions)
- 示例题库: 80 道入党积极分子考试真题
- Git 提交: 10+ 次, 迭代开发

2 AI 迭代对话记录

以下是开发过程中与 Claude AI 的关键对话摘录（按时间顺序）。

2.1 需求分析与架构设计

对话 1: 确定技术方案

用户: 阅读项目.md 完成项目开发，最后上传到 GitHub 让人可以在公网访问（要求用纯 C++ 开发，不要 C 语言，不需要 Python，提供一个题库，项目支持前后端，最后整合到一个.html 文件）

Claude: 提出三种架构方案：

A（推荐） C++ 后端 + 单文件.html 前端，本地运行服务，浏览器打开即用

B 纯前端 JS + localStorage，不需要后端

C C++ 编译为 WebAssembly 嵌入 HTML

用户选择: A

决策: 采用 C++17 + cpp-httplib + SQLite3 架构，前端 Bootstrap 5 单文件 SPA

对话 2: 需求细化

Claude: AI 功能在 C++ 中如何实现？（方案 A 保留完整/AI 降级/暂时砍掉）

用户选择: A —保留完整的 AI 功能

Claude: 题库 PDF/DOCX 解析方案选择

用户选择: A —预转为 JSON/TXT 格式导入

结论: AI 用 libcurl 对接 OpenAI 兼容 API；题库 JSON 格式导入，PDF 不直解

对话 3: 设计确认

Claude (呈现完整设计): 目录结构、API 路由、数据库 schema、前端页面规划

用户: 「可以，记得同时修改.md」

决策: 设计通过，启动分任务代理执行

2.2 开发过程中的代码审查与修复

对话 4: 数据库层代码审查发现 Bug 并修复

代码审查发现的问题:

1. `bindText` 函数把空字符串转成 `NULL`, 覆盖列默认值
2. 错题表缺少 `UNIQUE` 约束, `UPSERT` 不是原子操作 (`SELECT+INSERT` 竞态)
3. `answersMatch` 中迭代器跨容器比较—未定义行为 (UB)

修复:

- 182 移除 `bindText` 空串 $\rightarrow$ `NULL` 转换
- 182 添加 `ON CONFLICT(question_id) DO UPDATE SET` 原生 `UPSERT`
- 182 将 `trim` 结果绑定为具名变量, 消除 UB

对话 5: 编译修复—解决 Windows 环境问题

编译问题:

1. CMake 找不到 `libcurl` 开发包
2. `httplib.h` 中 `CreateFile2` 在 MinGW 未声明
3. 端口 8080 被旧 Python 服务占用

解决:

- 182 从 CDN 下载 `curl` 头文件 + 用 `gendef/dlltool` 生成导入库
- 182 `CreateFile2` $\rightarrow$ `CreateFileW` 适配 MinGW API
- 182 `taskkill` 关闭旧进程, 释放端口

对话 6: 公网部署问题排查

问题: GitHub Pages 返回 404

排查过程:

1. 检查仓库 $\rightarrow$ 正常
2. 检查 Pages 设置 $\rightarrow$ 正确
3. 发现: 仓库是 `Private`, GitHub Pages 需公开仓库 (免费版)

解决: 改仓库为 `Public` $\rightarrow$ 重新配置 Pages $\rightarrow$ 部署成功

最终成果: <https://chenyanzou.github.io/quiz-assistant/>

对话 7: C++ 现代化重构

用户: 「优化一下代码用 C++ 替换不要用 C 语言」

重构内容:

- `char*` 手动释放 → `std::unique_ptr` RAII
- `static` 文件级函数 → 匿名 namespace
- CURL 句柄手动管理 → `CurlHandle` / `CurlSlistGuard` RAII 类
- `reinterpret_cast` → `static_cast`
- `const char*` → `std::string_view`
- 函数声明 → 尾置返回类型 (C++11/17 风格)
- 未使用 `include` 清理

3 核心算法分析

3.1 HTTP 路由分发算法

核心算法

实现文件: quiz_server.cpp (cpp-httplib 框架内置)

cpp-httplib 使用正则匹配进行路由分发:

```
1 // 精确匹配
2 svr.Get("/api/categories", handler);
3 // 正则捕获匹配 (R"(...)" 为C++11原始字符串)
4 svr.Delete(R"(/api/categories/(\d+))", handler);
5 svr.Put(R"(/api/wrong/(\d+)/review)", handler);
```

算法: 遍历已注册路由表, 对每个路由用 `std::regex_match` 匹配请求路径。时间复杂度 $O(n \cdot m)$, n 为路由数量, m 为每个正则的匹配时间。

优化: 对 GET/POST 等方法分表存储, 减少匹配范围。

3.2 题目随机打乱算法 (Fisher-Yates Shuffle)

核心算法

实现文件: quiz_server.cpp:466-470

```
1 // Fisher-Yates 洗牌算法
2 std::random_device rd;           // 真随机种子
3 std::mt19937 g(rd());           // Mersenne Twister 19937 伪随机数
    生成器
4 std::shuffle(questions.begin(), questions.end(), g);
```

时间复杂度: $O(n)$

空间复杂度: $O(1)$ (原地打乱)

为什么不用 `rand()`: C 标准库 `rand()` 周期短 (2^{31}), 分布不均匀。C++17 的 `std::mt19937` 是业界标准的梅森旋转算法, 周期高达 $2^{19937} - 1$, 保证了题目的真正随机性。

3.3 答案比对算法（大小写不敏感 + 去空白）

核心算法

实现文件: quiz_server.cpp:48-55

```
1 [[nodiscard]] auto trim(std::string_view s) -> std::string {
2     auto start = s.find_first_not_of(" \t\n\r");
3     if (start == std::string_view::npos) return {};
4     auto end = s.find_last_not_of(" \t\n\r");
5     return std::string{s.substr(start, end - start + 1)};
6 }
7 // 使用 std::equal 四迭代器版本 + lambda 比较器
8 return std::equal(tu.begin(), tu.end(), tc.begin(), tc.end(),
9     [](char a, char b) {
10         return std::tolower(static_cast<unsigned char>(a))
11             == std::tolower(static_cast<unsigned char>(b));
12     });
```

时间复杂度: $O(n)$, n 为字符串长度

设计要点:

- unsigned char 转换避免中文等多字节字符的 UB
- 四迭代器版 std::equal 自动处理不同长度字符串
- string_view 避免不必要的拷贝

3.4 SQL 动态查询构建算法

核心算法

实现文件: database.cpp:196-240

查询构建采用条件串联 + 参数化模式:

```
1 std::string sql = "SELECT ... FROM questions WHERE 1=1";
2 if (category_id.has_value()) sql += " AND category_id = ?";
3 if (type.has_value())       sql += " AND question_type = ?";
4 if (search.has_value())     sql += " AND content LIKE ?";
5 sql += " ORDER BY id LIMIT ? OFFSET ?";
```

关键设计: 所有用户输入通过 `sqlite3_bind_*` 绑定, 而非字符串拼接, 从根本上防止 SQL 注入攻击。WHERE 1=1 作为锚点简化条件拼接逻辑。时间复杂度取决于数据库查询计划 (B 树索引扫描, 通常 $O(\log n)$)。

4 数据结构应用分析

4.1 核心数据结构一览

数据结构	应用位置	作用	算法复杂度
<code>std::vector<T></code>	题目列表、科目列表、JSON 数组	动态数组，随机访问	$O(1)$ 复杂度
<code>std::string</code>	题干内容、答案、解析	字符串存储与操作	—
<code>std::string_view</code>	trim, answersMatch	零拷贝子串引用	$O(1)$ 复杂度
<code>std::optional<T></code>	API 查询参数、数据库查询结果	可能缺失的值（替代 NULL）	$O(1)$ 复杂度
<code>std::unique_ptr<T></code>	句柄管理、资源所有权	RAII 自动释放	$O(1)$ 复杂度
<code>std::set<int></code>	错题去重 (quiz_server:448)	红黑树，有序不重复	$O(\log n)$ 复杂度
<code>std::regex</code>	HTTP 路由匹配	正则表达式模式匹配	—
<code>std::mt19937</code>	随机刷题	梅森旋转伪随机数	极高周期
B 树 (SQLite 隐式)	数据库索引	快速查找、排序、聚合	$O(\log n)$ 复杂度
Hash 表 (SQLite 隐式)	SQL 查询优化	JOIN 加速、去重	$O(1)$ 复杂度

表 2: 项目中使用的数据结构总览

4.2 课程知识点对照

数据结构与算法应用

与《C++ 数据结构与算法》课程对应：

- 向量 (vector)** —— 第 3/4 章：线性表与动态数组
在整个项目中作为核心容器使用，存放题目列表、科目列表、JSON 数组、API 响应等。
- 字符串与串匹配** —— 第 5 章：串与模式匹配
trim() 使用 find_first_not_of 和 find_last_not_of 进行字符集匹配；SQL LIKE 查询实现模糊搜索（本质是 BFSM 模式匹配）。
- 集合 (set) 与红黑树** —— 第 8 章：树与二叉树
错题重做模式中用 std::set<int> 去重，底层是红黑树，保证已见题目集合的有序与唯一。
- 散列表 (Hash)** —— 第 9 章：查找
SQLite 内部使用 B 树和 Hash 表优化索引查询、JOIN 关联和 GROUP BY 聚合；API 响应构建中隐式使用 nlohmann::json（内部为红黑树/哈希混合）。
- 排序算法** —— 第 10 章：排序
Fisher-Yates std::shuffle 原地随机化 ($O(n)$)；SQLite ORDER BY 使用 B 树/B+ 树索引排序 ($O(n \log n)$)。

6. 递归与分治——第 2 章：递归

数据库递归查询 (SQLite WITH RECURSIVE 在本项目中用于统计子查询); JSON 对象树的递归解析 (nlohmann/json 内部实现)。

7. 资源管理 (RAII) ——贯穿全书

C++ 核心编程范式: StmtGuard、CurlHandle、CurlSlistGuard、Database 类全部基于 RAII, 确保语从句柄、网络连接、数据库连接在作用域结束时自动释放。

5 工程能力提升总结

5.1 AI 辅助开发的经验与反思

5.1.1 AI 生成代码的优点

1. **大幅提升开发速度**—AI 在几分钟内生成了 2000 行的 C++ 代码骨架和完整的数据库层实现
2. **减少重复劳动**—18 个 API 路由的模板代码、SQL 语句编写、JSON 解析等机械性工作由 AI 完成
3. **提供多种设计方案**—在前期讨论了 3 种架构方案，帮助快速决策
4. **擅长大规模代码生成**—前端 HTML (2144 行) 可基本由 AI 完成

5.1.2 AI 生成代码的局限性

1. **存在潜在 Bug**—代码审查发现了多处问题：
 - `answersMatch` 中迭代器来自两个不同临时对象 → 未定义行为
 - `bindText` 空串 → NULL 导致数据库默认值被覆盖
 - 错题 Upsert 非原子操作 → 竞态条件
2. **缺乏领域知识**—AI 不了解特定平台限制 (MinGW 无 `CreateFile2`、CMake 编码问题、`libcurl` 无开发包时的处理)
3. **需要人工验证**—所有 AI 生成的代码都需要经过代码审查、编译测试、运行时测试
4. **架构理解受限**—AI 无法完全把握全局设计意图，容易出现“只见树木不见森林”的问题

5.1.3 人工算法的必要性

AI 不能替代的内容:

- 根据课程要求设计符合教学目的算法实现 (如答案比对中的 `trim` + 大小写不敏感比较)
- RAII 资源管理范式的全局设计决策
- 特定平台问题的调试与解决 (Windows SSL、CMake 路径编码)
- 代码质量的审查和算法正确性的验证
- 将课程理论知识 (B 树、红黑树、哈希表) 映射到实际项目中的能力

5.2 个人能力提升

1. **C++ 工程化能力**—从零搭建 C++ 项目，理解 CMake 构建系统、第三方库集成、链接过程

2. **数据结构应用**—将课本上的 `vector`、`set`、`string_view`、`optional` 等知识点应用到实际项目中
3. **数据库设计**—从 ER 模型到 SQL 建表、索引优化、SQL 注入防护的完整实践
4. **RAII 设计模式**—掌握了资源获取即初始化的核心 C++ 编程范式
5. **调试能力**—面对未定义行为 (UB)、平台兼容性问题、网络端口冲突等实际问题的排查经验
6. **代码审查意识**—学会了审查 AI 生成的代码，识别潜在 Bug、安全问题和设计缺陷
7. **全栈开发思维**—理解了前后端分离架构、RESTful API 设计、Git 版本管理、CI/CD 自动部署
8. **文档撰写能力**—编写规范的设计文档 (Spec)、实施计划 (Plan)、项目总结报告 (本文)

6 项目总结

6.1 完成情况

交付物	状态	说明
C++ 后端服务	✓	860 行, 18 个 REST API
SQLite 数据库层	✓	4 张表, 完整的 CRUD+ 统计
AI 多厂商服务	✓	OpenAI/Kimi/DeepSeek/Qwen/GLM/Ollama
HTML 单文件前端	✓	7 页面 SPA, 80 题内嵌
GitHub 仓库	✓	chenyanzou/quiz-assistant
公网部署 (GitHub Pages)	✓	https://chenyanzou.github.io/quiz-assistant/
C++ 现代化重构	✓	RAII 全覆盖, 纯 C++17 风格
设计文档	✓	docs/superpowers/specs/
实施计划	✓	docs/superpowers/plans/
项目总结报告	✓	本文档

表 3: 交付物清单

6.2 工程反思

1. **测试先行**—首次开发时没有编写单元测试, 导致后期在编译和运行时才发现问题。后续项目应当采用 TDD 方式。
2. **依赖管理**—Windows 环境下 C++ 第三方库的获取和集成是一大痛点, CMake + vcpkg/Conan 可以改善体验。
3. **平台兼容性**—MinGW vs MSVC 的 API 差异 (如 CreateFile2 vs CreateFileW) 提醒我们在跨平台开发时需要做好抽象层。
4. **安全第一**—SQL 注入防护 (参数化查询)、输入校验 (trim、类型检查)、API 密钥保护 (localStorage 不复传) 都是本项目实际做了的安全措施。

6.3 致谢

感谢《C++ 数据结构与算法编程》课程提供的理论与实践基础, 感谢 Claude AI 作为编码助手在开发过程中的辅助, 让我在大一阶段就完成了一个完整的全栈 C++ 项目。